

系统架构设计师

数据库

姜美荣



目录

直播内容：案例之数据库

- 规范化与反规范化★★★★
- 索引与视图★★★★
- 分布式数据库★★★★
- NoSQL★★★★
- 内存数据库★★★★
- 数据库的优化★★★★
- redis★★★★

数据库考试内容

年份	主题	分值
2026上	悲观锁，乐观锁，Redis高并发的控制场景的优化策略	25
2025下	redis互斥锁缓存实现方案、逻辑过期方案、两方案对比	25
2025上	redis主从同步填空、增量主从同步填空、redis数据库持久化的方式对比	25
2024年下	数据库设计-Cache-aside架构、数据库读写、一致性问题	25分
2024年上	数据库设计-分布式锁、Redis锁、死锁场景、ZSET命令	25分
2023年	数据库设计-数据持久层，Redis，读写分离	25分
	大数据架构	25分
2022年	分布式数据库缓存设计-一致性、布隆过滤器	25分
2021年	数据库设计-规范化与反规范化技术、Redis	25分
2020年	分布式数据库缓存设计-Redis	25分
	数据库设计-规范化数据库设计	25分
2019	分布式数据库缓存设计-一致性问题 and 运维	25分
	Web系统架构设计-SQL注入攻击	7分
2018	分布式数据库缓存设计-MemCache和Redis	25分

数据库考试内容

本章为核心章节，须反复练习。

单选：4分左右

案例：有所涉及

基本概念：三级模式-两级映像、关系代数运算

数据库模型：E-R模型、关系模型、关系代数（结合SQL语言）

规范化：函数依赖、键与约束、范式、模式分解

事务并发：并发三种问题、三级封锁协议

数据库的设计：规划、需求分析、概念设计、逻辑设计、物理设计。各阶段的可交付成果。

数据库的性能优化：硬件、设计、应用软件、系统软件。

数据库新技术：数据库安全与备份、反规范化、分布式数据库、缓存数据库、数据库集群、

两阶段提交、三阶段提交

Redis:主从复制、同步机制、三种模式、持久化方式、锁机制

数据库设计

- (1)需求分析：即分析数据存储的要求，产出物有数据流图、数据字典、需求说明书。获得用户对系统的三个要求：信息要求、处理要求、系统要求。
- (2)概念结构设计：就是设计E-R图，也即实体-联系图。工作步骤包括：**选择局部应用、逐一设计分E-R图、E-R图合并。**
- 分E-R图进行合并时，它们之间存在的冲突主要有以下3类。
- 属性冲突。同一属性可能会存在于不同的分E-R图中。
 - 命名冲突。相同意义的属性，在不同的分E-R图上有着不同的命名，或是名称相同的属性在不同的分E-R图中代表着不同的意义。
 - 结构冲突。同一实体在不同的分E-R图中有不同的属性，同一对象在某一分E-R图中被抽象为实体而在另一分E-R图中又被抽象为属性。
- (3)逻辑结构设计：将E-R图，转换成关系模式。工作步骤包括：**确定数据模型、将E-R图转换成为指定的数据模型、确定完整性约束和确定用户视图、规范化设计、优化。**
- (4)物理设计：步骤包括确定**数据分布、存储结构和访问方式。**
- (5)数据库实施阶段。根据逻辑设计和物理设计阶段的结果建立数据库，**编制与调试应用程序，组织数据入库，并进行试运行。**
- (6)数据库运行和维护阶段。数据库应用系统经过试运行即可投入运行，但该阶段需要不断地对系统进行评价、调整与修改。数据库维护工作的主要内容包括**对数据库性能的监测和改善、故障恢复、数据库的重组和重构。**在数据库运行阶段，对数据库的维护主要由**DBA** 完成。

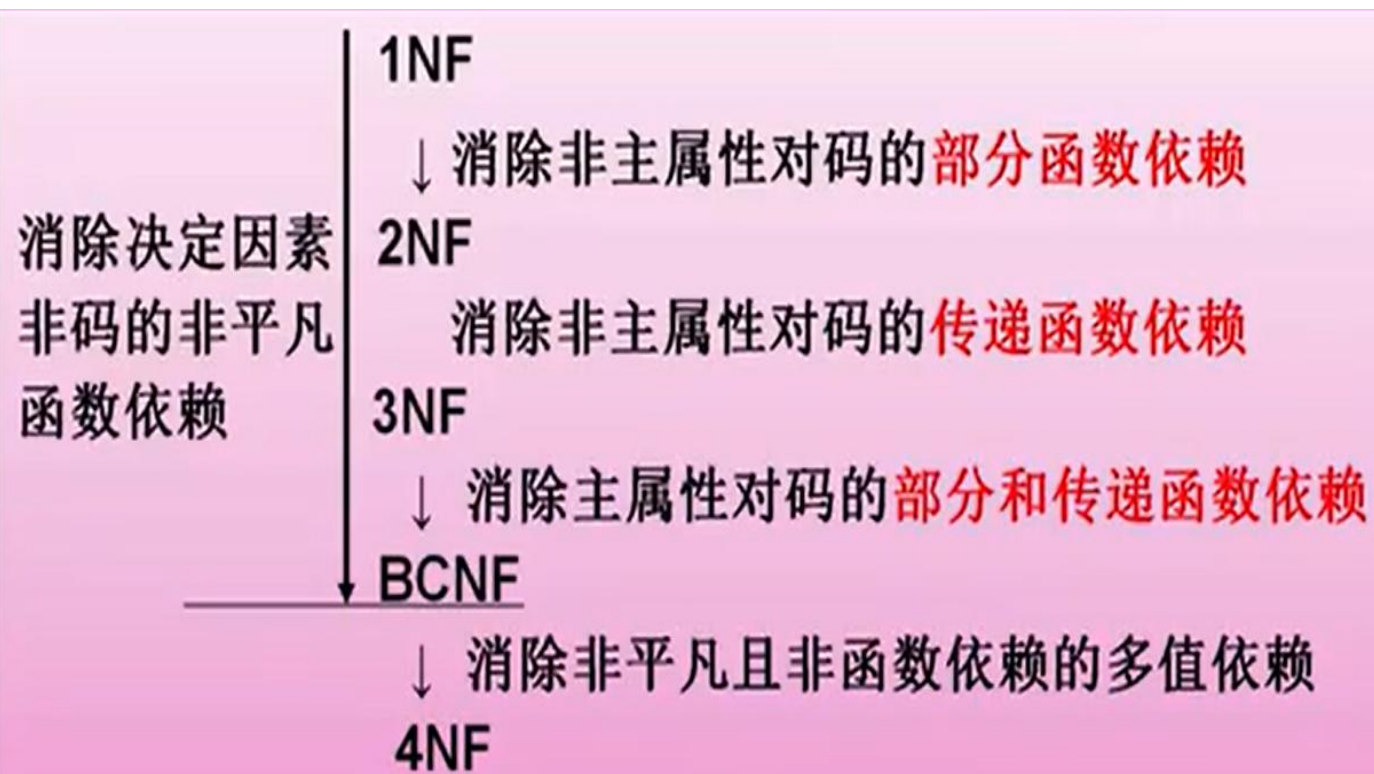
规范化理论

不规范化带来的四大问题：

设有一个关系模式R (SNAME, CNAME, TNAME TADDRESS)，其属性分别表示学生姓名、选修的课程名、任课教师姓名和任课教师地址。仔细分析一下，就会发现这个模式存在下列存储异常的问题：

- (1) **数据冗余：**数据被重复存储，如某门课程有100个学生选修，那么在R的关系中就要出现100个元组，这门课程的任课教师姓名和地址也随之重复出现100次。
- (2) **修改异常：**修改导致数据不一致，如由于上述冗余问题，当需要修改这个教师的地址时，就要修改100个元组中的地址值，否则就会出现地址值不一致的现象。
- (3) **插入异常：**插入时异常，如不知道听课学生名单，这个教师的任课情况和家庭地址就无法进入数据库；否则就要在学生姓名处插入空值。
- (4) **删除异常：**删除了不该删除的数据，如当只有一条记录时，要删除这个学生选课信息，会将课程名、教师名和教师地址都给删除了。

规范化理论



反规范化

反规范化是加速读操作性能(数据检索)的方法，通过有选择地在数据结构标准化后添加特定的冗余数据来增加查询效率，牺牲部分规范化来提高性能。

益处：连接操作少，**检索快、统计快**，需要查的表减少，检索容易。

挑战：空间占用大，操作开销大，数据不一致，更新和插入代码更难。

常见的反规范化操作有：**冗余列、派生列、表重组和表分割**，其中表分割又分为水平分割和垂直分割。

增加冗余列	在多个表中具有相同的列，它常用来在查询时避免连接操作。
增加派生列	增加的列可以通过表中其他数据计算生成。
重新组表	如果许多用户需要查看两个表连接出来的结果数据，则把这两个表重新组成一个表来减少连接而提高性能。
水平分割表	按记录进行分割，把数据放到多个独立的表中。
垂直分割表	对表进行分割，将主键与部分列放到一个表中，主键与其它列放到另一个表中。

反规范化的缺点	解决方案
数据冗余，需要更大存储空间	无解
插入、更新、删除操作开销更大	无解
数据不一致 可能产生添加、修改、删除异常	1、触发器数据同步 2、应用程序数据同步 3、批处理
更新和插入代码更难写	无解

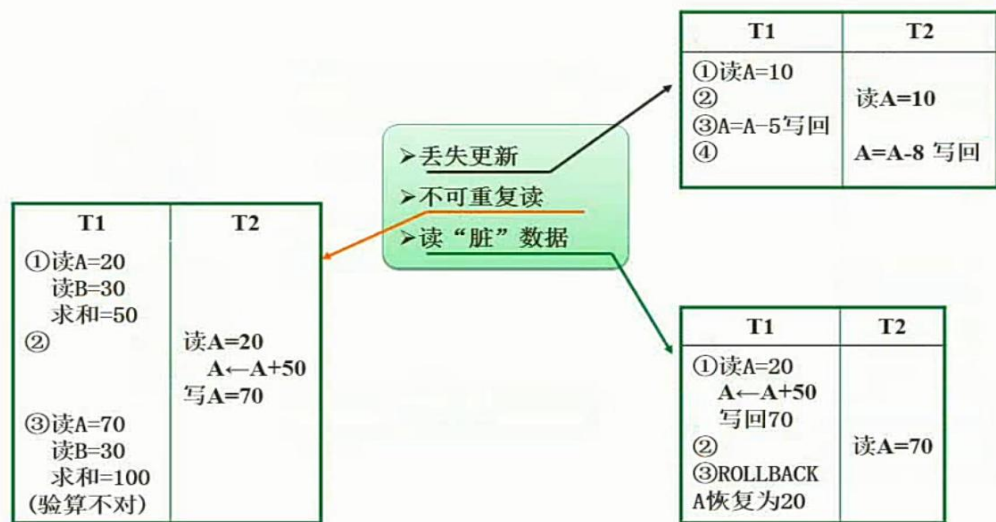
反规范化



并发控制：事务管理

事务通常以 BEGIN TRANSACTION（事务开始）语句开始，以 COMMIT 或 ROLLBACK 语句结束。COMMIT 称为“事务提交语句”，表示事务执行成功的结束。ROLLBACK 称为“事务回退语句”，表示事务执行不成功的结束。

- **丢失更新**：事务1对数据A进行了修改并写回，事务2也对A进行了修改并写回，此时事务2写回的数据会覆盖事务1写回的数据，就丢失了事务1对A的更新。即对数据A的更新会被覆盖。
- **不可重复读**：事务2读A,而后事务1对数据A进行了修改并写回，此时若事务2再读A，发现数据不对。即一个事务重复读A两次，会发现数据A有误。
- **读脏数据**：事务1对数据A进行了修改后，事务2读数据A，而后事务1回滚，数据A恢复了原来的值，那么事务2对数据A做的事是无效的，读到了脏数据。



并发控制-事务管理-封锁技术

处理并发控制的主要方法是采用封锁技术。

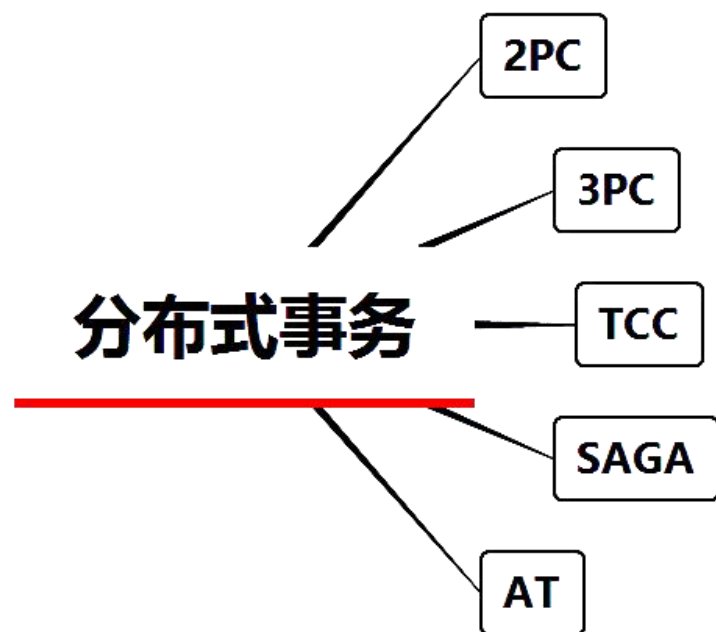
它有两种类型：排他型封锁（X 封锁）和共享型封锁（S 封锁），分别介绍如下：

（1）排他型封锁（简称 X 封锁）。如果事务 T 对数据 A（可以是数据项、记录、数据集，乃至整个数据库）实现了 X 封锁，那么只允许事务 T 读取和修改数据 A，其他事务要等事务 T 解除 X 封锁以后，才能对数据 A 实现任何类型的封锁。可见 X 封锁只允许一个事务独锁某个数据，具有排他性。

（2）共享型封锁（简称 S 封锁）。X 封锁只允许一个事务独锁和使用数据，要求太严。需要适当放宽，例如可以允许并发读，但不允许修改，这就产生了 S 封锁概念。S 封锁的含义是：如果事务 T 对数据 A 实现了 S 封锁，那么允许事务 T 读取数据 A，但不能修改数据 A，在所有 S 封锁解除之前绝不允许任何事务对数据 A 实现 X 封锁。

独占X，共享S

分布式数据库管理系统



分布式数据库管理系统

两阶段提交协议 (★★)

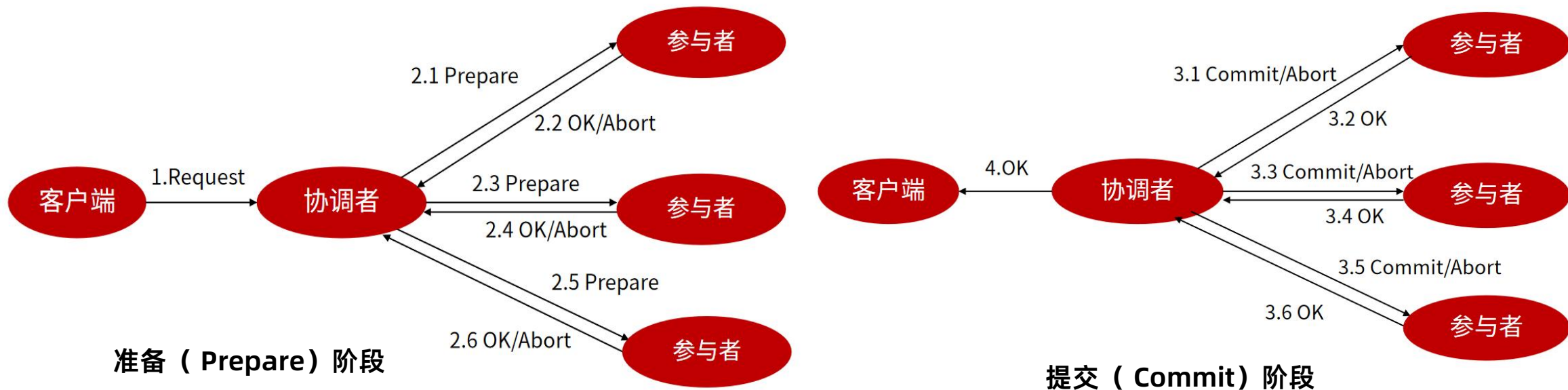
2PC协议把数据提交分成两个阶段

n 表决阶段：协调者发布“准备提交”指令，所有参与者进行表决，具有一票否决权.目的是形成一个共同的决定。

n 执行阶段：根据协调者的指令，参与者提交或撤销事务，并给协调者发送确认信息。目的是实现这个协调者的决定。

规则：只要有一个参与者撤销事务，协调者就必须做出全局撤销的决定。

只有所有参与者都同意提交事务，协调者才能做出全局提交的决定。

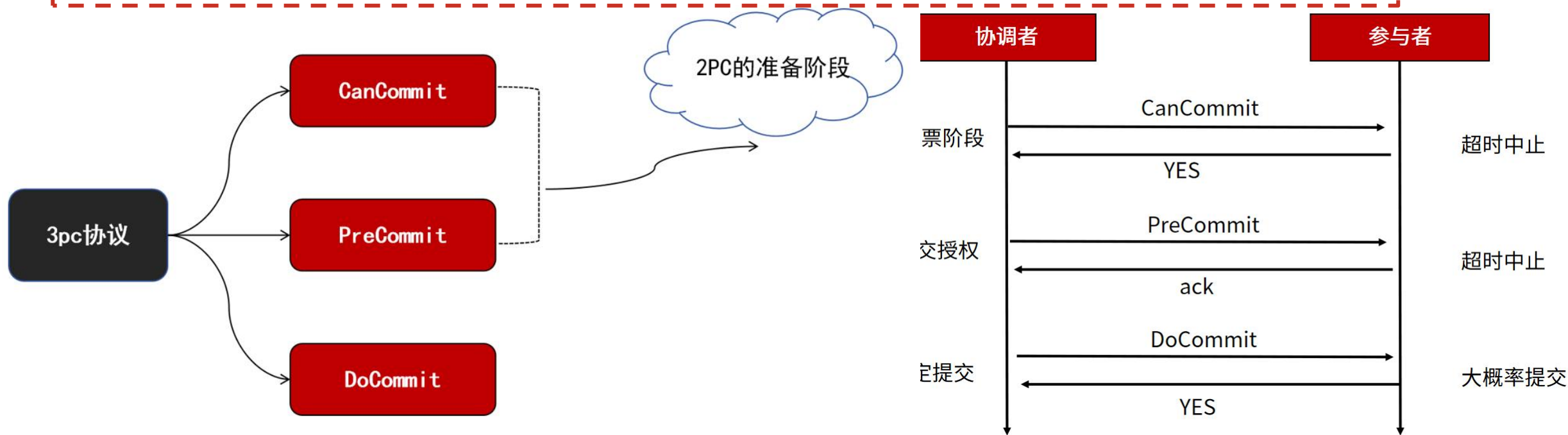


分布式数据库管理系统

三阶段提交协议

3PC协议把数据提交分成两个阶段

- n 第一阶段：协调者向所有参与者发布“准备提交”指令，所有参与者进行表决，只有所有参与者都投票“建议提交”，才会进入第二阶段。
- n 第二阶段：协调者向所有参与者发“全局预提交”，只有所有参与者都提交“准备就绪”才能进入第三阶段
- n 第三阶段：协调者向所有的参与者发布“全局提交”报文

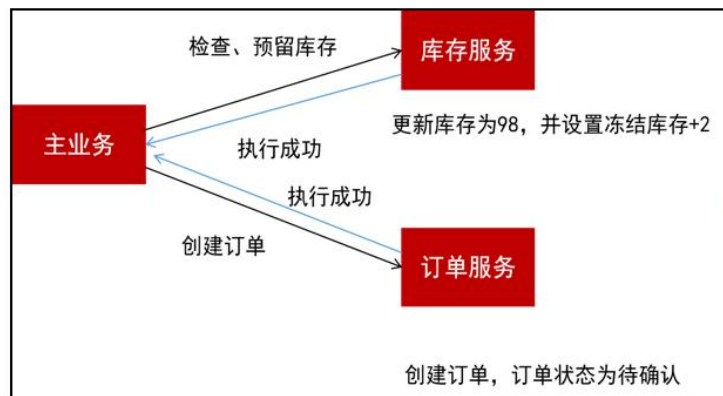
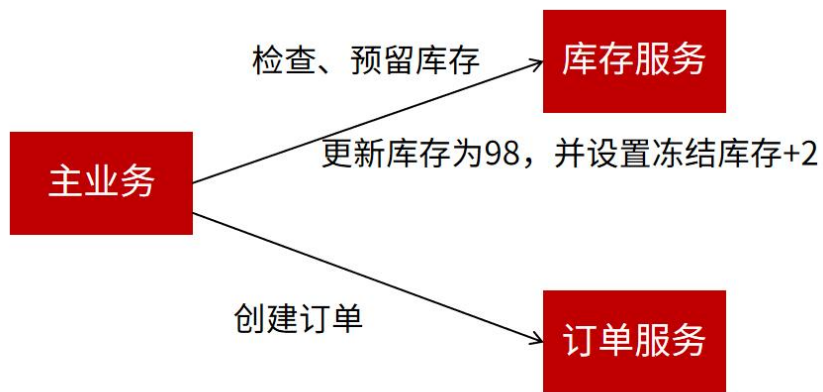


分布式数据库管理系统

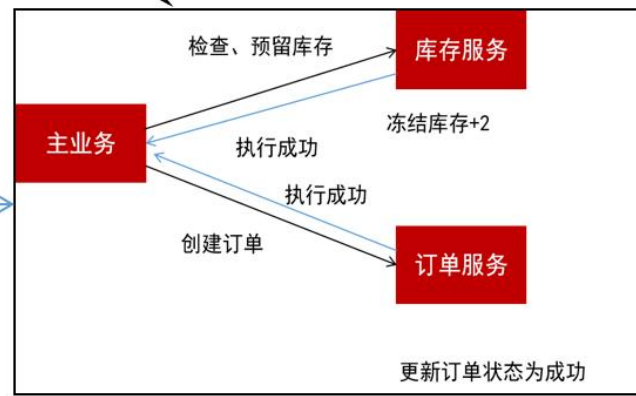
特征	两阶段提交 (2PC)	三阶段提交 (3PC)
阶段划分	准备阶段和提交阶段	CanCommit阶段、PreCommit阶段和DoCommit阶段
超时机制	仅协调者有超时机制	协调者和参与者均有超时机制
阻塞问题	可能导致长时间阻塞	引入额外的尝试阶段和超时机制, 减少阻塞可能性
复杂性	较简单	较复杂

分布式数据库管理系统

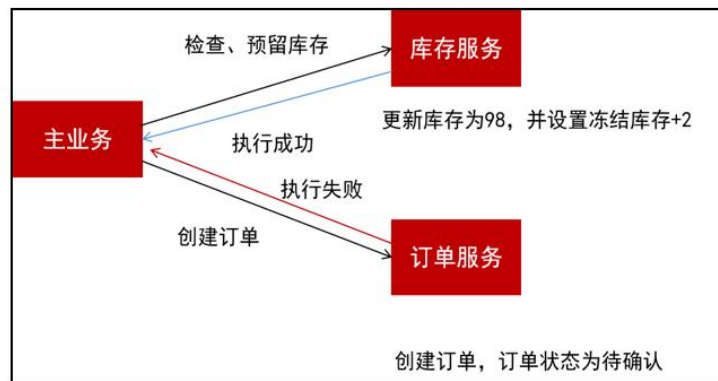
TCC模式：



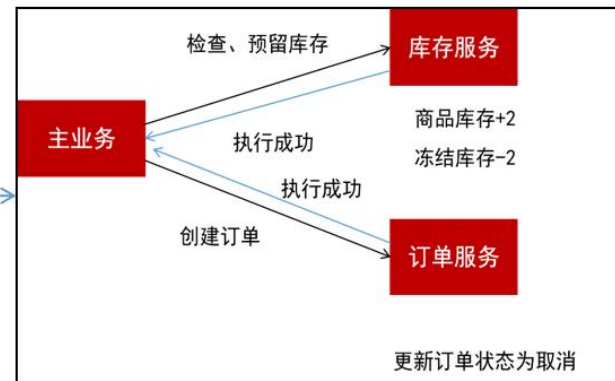
Confirm



失败情况下



Cance



分布式数据库管理系统

Ø TCC模式的特点：

- l 非阻塞事务：相比传统的两阶段提交（2PC）模式，TCC模式在第一阶段（Try阶段）完成后，不阻塞其他事务的执行，提高了系统的并发性能。
- l 性能提升：由于避免了长时间的事务锁定，TCC模式在高性能、高并发的场景下具有更好的表现。减少了数据库锁的竞争，降低了死锁的风险。
- l 灵活性和细粒度控制：开发者可以精确控制每个事务阶段的行为，更好地适应复杂的业务逻辑。Try、Confirm、Cancel三个阶段的逻辑完全由开发者实现，提供了高度的灵活性。
- l 业务补偿机制：通过Confirm和Cancel阶段的业务补偿操作，确保事务的最终一致性。在出现异常时，能够自动回滚预留的资源，保证数据的一致性。

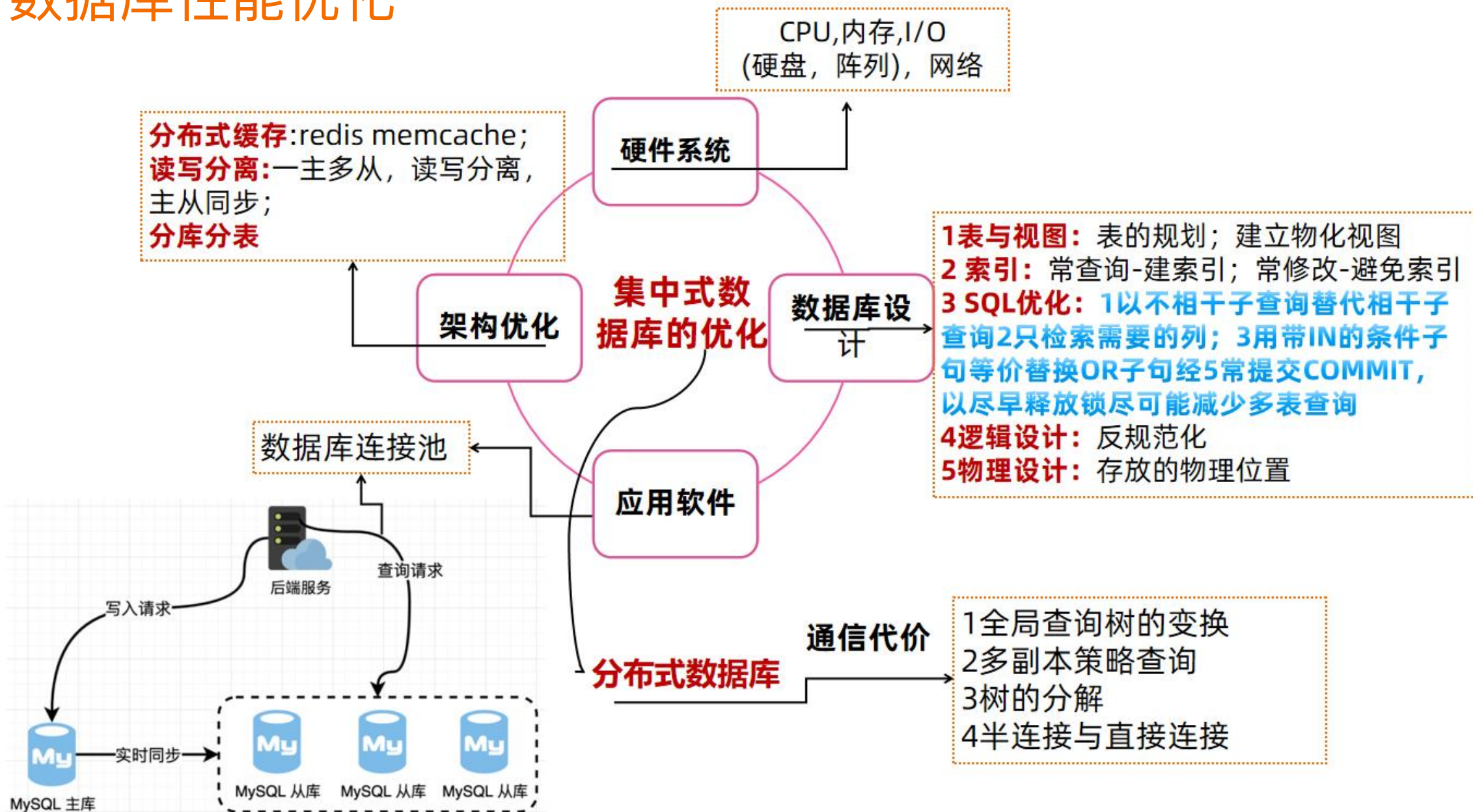
NOSQL VS 关系数据库

对比维度	关系数据库	NoSQL
应用领域	面向 通用 领域	特定 应用领域
数据容量	有限 数据	海量 数据
数据类型	结构化 数据二维表	非结构化 数据
并发支持	支持 并发 、但 性能低	高并发
事务支持	高 事务性	弱 事务性
扩展方式	向 上 扩展	向 外 扩展

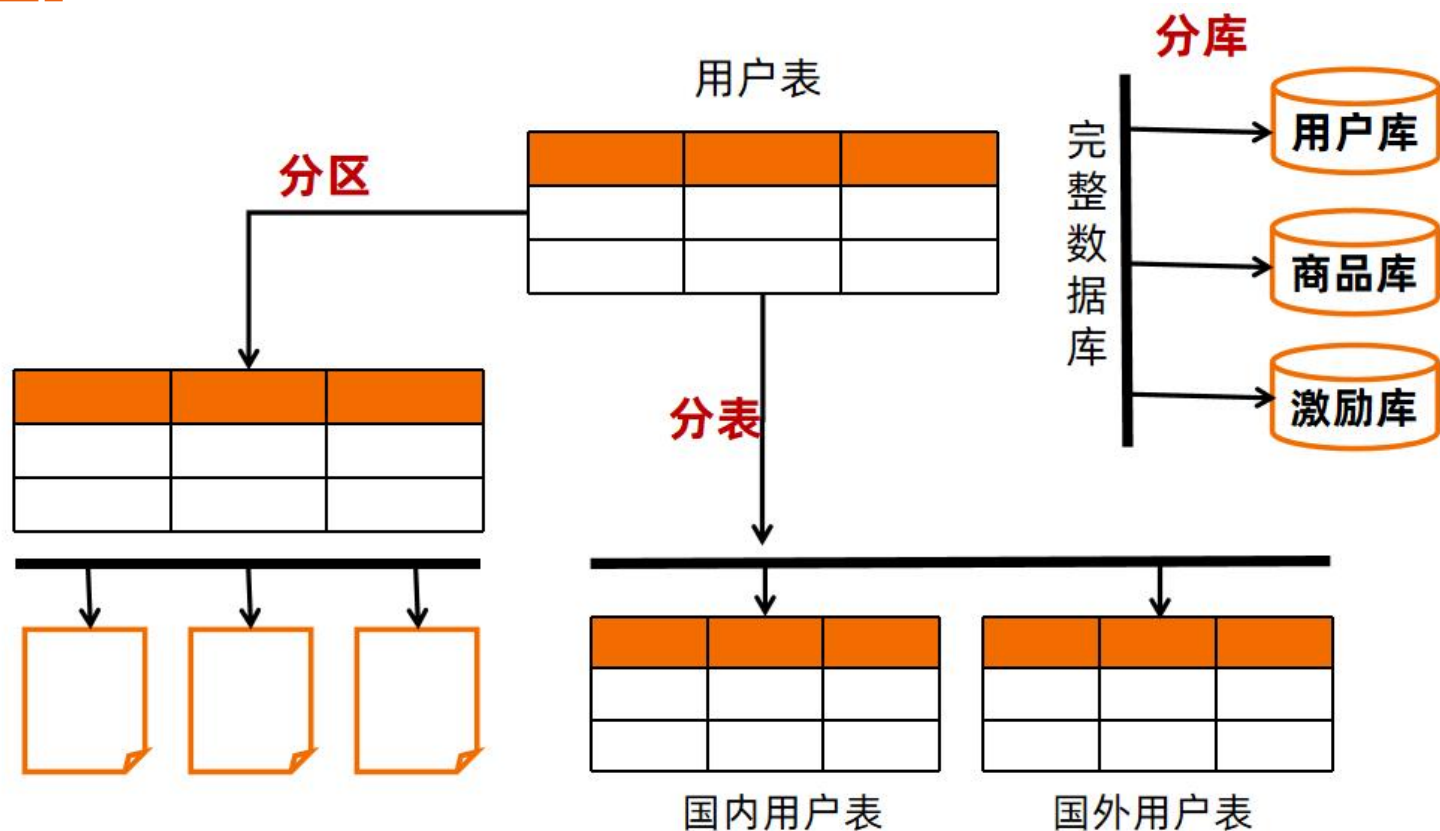
NoSQL 数据库

分 类	典型产品	应用场景	优 点	缺 点	数据模型
文档存储	MongoDB 、 CouchDB	Web应用，存储面向文档和半结构化数据	结构灵活，可以根据value构建索引	缺乏统一的查询语法，无事务处理能力	Key-Value对应的键值对，Value为结构化数据
键值存储	Memcached、 Redis	内容缓存，如会话、配置文件、参数等	扩展性好，灵活性强，大量操作时性能高	数据无结构化，通常被当成字符串或者二进制数据，通过键查询值	Key 指向 Value的键值对，通常用hash table来实现
列存储	BigTable、 HBase 、 Cassandra	分布式数据存储和管理	可扩展性强，查找速度快，复杂性低	功能局限，不支持事务的强一致性	以 列簇式存储 ，将同一列数据存在一起
图存储	Neo4j、 OrientDB	社交网络、推荐系统，专注于构建系统图谱	支持复杂的图形算法	复杂性高，只能支持一定的数据规模	图结构

数据库性能优化



分区分表分库



共性:

- 1、都针对数据表
- 2、都使用了分布式存储
- 3、都提升了查询效率
- 4、都降低了数据库的频繁I/O压力值

特性:

分区逻辑上还是一张表，
分表逻辑上已是多张表

分区分表分库-分区方法

Range (范围)

按照时间拆分

Hash之后按照分表个数取模

分区的优点

- 1、相对于单个文件系统或是硬盘，分区可以存储更多的数据。
- 2、数据管理比较方便，比如要清理或废弃某年的数据，就可以直接删除该日期的分区数据即可。
- 3、精准定位分区查询数据，不需要全表扫描查询，大大提高数据检索效率。
- 4、可跨多个分区磁盘查询，来提高查询的吞吐量。
- 5、在涉及聚合函数查询时，可以很容易进行数据的合并。

分区分表分库-分区方法

Range (范围)

按照时间拆分

Hash之后按照分表个数取模

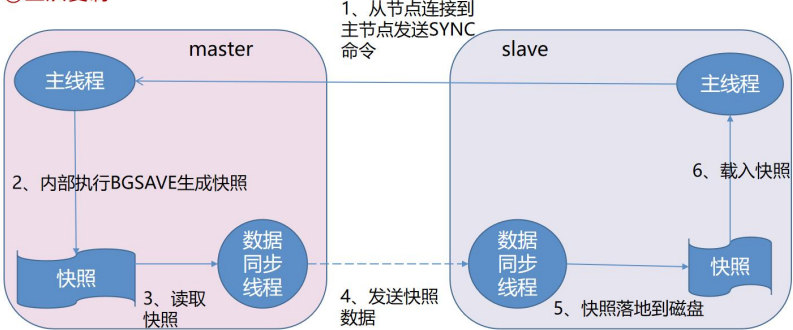
分区的优点

- 1、相对于单个文件系统或是硬盘，分区可以存储更多的数据。
- 2、数据管理比较方便，比如要清理或废弃某年的数据，就可以直接删除该日期的分区数据即可。
- 3、精准定位分区查询数据，不需要全表扫描查询，大大提高数据检索效率。
- 4、可跨多个分区磁盘查询，来提高查询的吞吐量。
- 5、在涉及聚合函数查询时，可以很容易进行数据的合并。

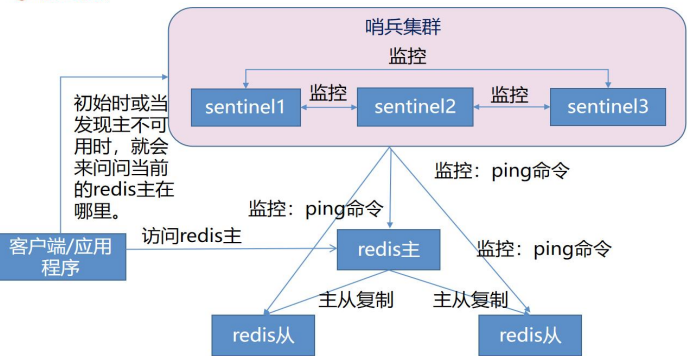
Redis分布式存储方案

Redis分布式存储方案	模式
主从模式	一主多从，故障时手动切换。
哨兵模式	有哨兵的一主多从，主节点故障 自动选择 新的主节点。
集群模式	分节点对等集群 ，分 slots ，不同slots的信息存储到不同节点

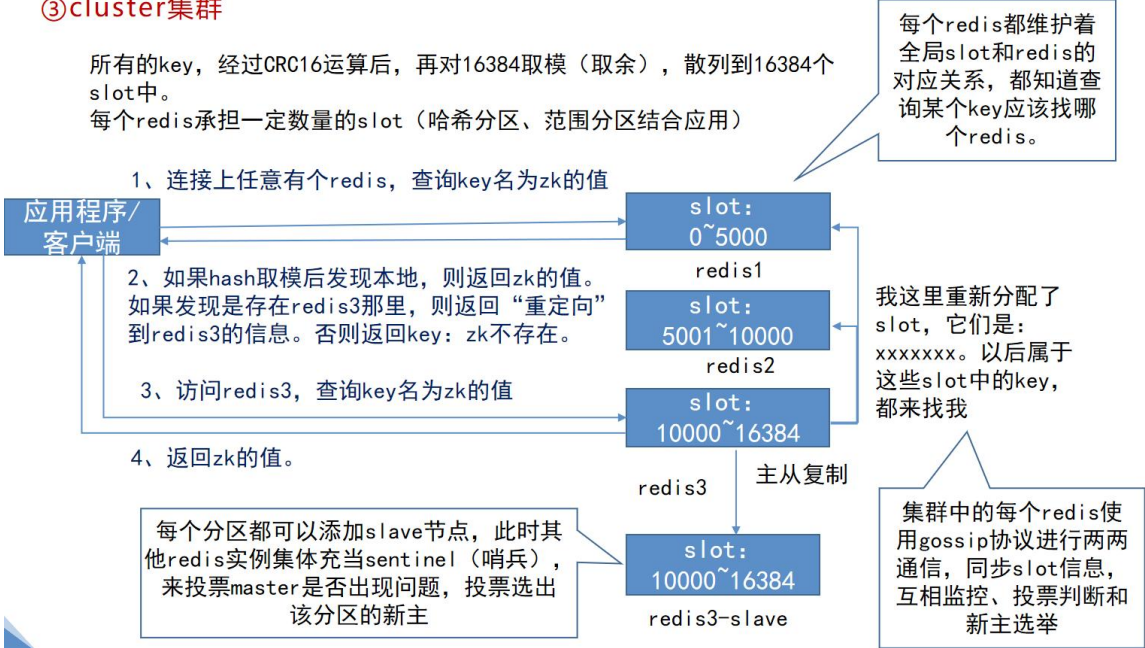
①主从复制



②哨兵集群



③cluster集群



Redis持久化存储的机制

Redis提供了两种持久化存储的机制，分别是**RDB (Redis DataBase)持久化方式**和**AOF (Append Only File)持久化方式**。一旦服务器宕机，内存中的数据将全部丢失。我们很容易想到的一个解决方案是，从后端数据库恢复这些数据，但这种方式存在两个问题：一是需要频繁访问数据库，会给数据库带来巨大的压力；二是这些数据是从慢速数据库中读取出来的，性能肯定比不上从 Redis 中读取，导致使用这些数据的应用程序响应变慢。所以，对 Redis 来说，实现数据的持久化，避免从后端数据库中进行恢复，是至关重要的。（★★）

两种持久化机制	RDB内存快照 (RedisDataBase)	AOF日志 (Append Only File)
说明	把当前内存中的数据快照写入磁盘（数据库中所有键值对数据）。恢复时是将快照文件直接读到内存里。	通过持续不断地保存Redis服务器所执行的更新命令来记录数据库状态，类似mysql的binlog。恢复数据时需要从头开始回放更新命令。
磁盘刷新频率	低	高
文件大小	小	大
数据恢复效率	高	低
数据安全	低	高

■ 典型真题 (202605架构真题)

某电商平台专注于日常购物场景。用户可浏览商品列表，通过分类筛选或关键词检索定位目标商品。选定商品后支持加入购物车，并可调整购买数量。购物车支持商品编辑与批量结算。结算时需填写收货信息，系统自动计算应付总额，支持微信、支付宝等支付方式。支付成功后生成订单，用户可查看订单状态与物流轨迹，确认收货后可提交评价。平台设有满减优惠、限时折扣等促销规则，结算时自动匹配最优优惠。另设秒杀专区，特定时段推出限量商品特价抢购，用户需在倒计时结束前完成支付。

【问题1】为保证秒杀的数据一致性，使用悲观锁进行并发访问控制。悲观锁按行为分类有几种，至少给出两种设计方案。

典型真题（202605架构真题）

【问题1】为保证秒杀的数据一致性，使用悲观锁进行并发访问控制。悲观锁按行为分类有几种，至少给出两种设计方案。

参考答案：

悲观锁按行为分排他锁与共享锁。实现方式是数据库行级排他锁与基于Redis的分布式互斥锁方案。

悲观锁按行为主要分为共享锁（S锁）和排他锁（X锁）。共享锁允许其他事务读但不允许改，排他锁则禁止其他事务读和改，从而实现完全独占。一般情况下会使用排他锁，针对于排他锁的两种设计方案如下：

1、数据库行级排他锁方案

在事务中通过 `SELECT ... FOR UPDATE` 对目标数据行加排他锁。例如扣减库存时，先锁定该商品库存记录，完成扣减后再提交事务释放锁。其他并发事务必须等待锁释放才能继续操作，有效防止超卖。

2、分布式互斥锁方案

使用 Redis 的 `SET` 命令配合 `NX` 和过期参数实现分布式锁，使用 `SET key value NX PX <milliseconds>` 或 `SET key value NX EX <seconds>` 命令。业务操作前先尝试获取锁，获取成功则执行后续逻辑，执行完毕释放锁；获取失败则自旋等待或直接返回错误。该方案适用于微服务架构下的跨进程并发控制。

典型真题 (202605架构真题)

【问题2】乐观锁的实现方式是什么，比较乐观锁和悲观锁的优缺点。

乐观锁：是一种并发控制策略，假设冲突很少发生，所以不加锁，只在提交时检查数据是否被别人改过。优点是无锁等待、吞吐量高、适合读多写少场景，缺点是冲突时需重试，高并发写入下失败率高。乐观锁的实现方式有版本号机制、时间戳机制和条件字段 CAS机制。

悲观锁：假设冲突多，操作前先加锁，优点是数据一致性强、适合写多场景，缺点是锁等待开销大、容易死锁、并发性能差。选哪个取决于冲突概率：冲突少用乐观锁，冲突多用悲观锁。

典型真题 (202605架构真题)

试题分析：

乐观锁不是数据库锁机制，而是在应用层通过冲突检测来保证数据一致性。常见实现方式有：

1、版本号机制：表增加 version 字段。更新时带上版本号条件，并同时自增版本号，通过判断受影响行数是否大于 0 来检测冲突。

例如，执行UPDATE product

```
SET stock = stock - 1, version = version + 1
```

```
WHERE id = 100 AND version = 5;
```

数据库会返回一个受影响的行数：受影响的行数 = 1：说明 WHERE 条件匹配到了记录，版本号一致，更新成功。受影响的行数 = 0：说明 WHERE 条件未匹配到记录，也就是数据在读取后，版本号已被其他事务修改，本次更新失败，发生了并发冲突。此时通常需要在业务代码中重试或直接返回失败。

2、时间戳机制：在数据表中增加一个最后修改时间字段，更新时必须在 WHERE 条件中带上读取时获得的时间戳，只有当时间戳未发生变化时才能成功更新，否则判定为冲突。例如，UPDATE product

```
SET stock = stock - 1, last_modified = NOW()
```

```
WHERE id = 100 AND last_modified = '2026-06-08 10:00:00';
```

执行后，通过检查 受影响的行数判断是否冲突：

若行数 = 1，说明时间戳未变，更新成功。

若行数 = 0，说明在读取之后，该行已被其他事务修改，此时发生并发冲突，需在代码中重试或返回失败。

3、条件字段 CAS：不依赖额外字段，而是直接将业务字段的旧值作为更新条件，例如扣减库存时写成 UPDATE ... WHERE stock = oldStock，若该字段的值已被其他事务修改则条件不成立，从而检测冲突。

典型真题 (202605架构真题)

乐观锁与悲观锁的比较如下：

对比维度	悲观锁	乐观锁
核心思想	假定冲突总会发生，先锁住再操作，确保独占。	假定冲突不会发生，先操作，提交时检查是否被改动过。
实现机制	数据库行锁、分布式锁等，阻塞其他事务/线程。	版本号/时间戳更新时带条件判断并自增版本。
冲突处理	直接等待或失败，拿不到锁就阻塞。	先更新，发现版本号不一致则更新失败，通常需要重试。
并发性	并发度较低，锁持有期间别人只能等，适合冲突多的写场景。	并发度高，无阻塞，但冲突多时重试成本高，适合读多写少。
一致性	强一致	最终一致

典型真题 (202605架构真题)

【问题3】基于Redis缓存给出至少三种高并发的控制场景的优化策略。

- 1、页面缓存：秒杀开始前，把活动页面的 HTML 整页缓存到 Redis，用户请求直接返回缓存页面，根本不查数据库。
- 2、数据缓存 + 原子扣减：库存从数据库搬到 Redis，用 Lua 脚本一次性完成"查库存 + 扣库存 + 防重复"三件事，保证原子性，然后异步落库。
- 3、分布式锁：用 Redis 的 SETNX + 过期时间保证同一时间只有一个请求能操作库存。

■ 典型真题 (202605架构真题)

- 1、页面缓存：秒杀开始前，提前把活动页面的 HTML 整页写入 Redis，设置较长的过期时间。用户打开秒杀页面时，请求直接命中 Redis 缓存，返回静态页面，连数据库都不会碰。这样做的核心目的是挡住绝大多数"只是看看"的流量，让真正的秒杀请求尽可能地到达后端服务，从源头上降低系统压力。
- 2、数据缓存 + 原子扣减：库存数据不再每次查数据库，而是提前加载到 Redis 中。用户下单时，用一段 Lua 脚本在 Redis 里一次性完成三件事：判断库存是否足够、扣减库存、记录用户是否已抢过。Lua 脚本在 Redis 中是原子执行的，不会被其他请求打断，所以不需要额外加锁就能保证数据一致。扣减成功后，再通过消息队列异步写入数据库，既保证了高并发下的正确性，又不让数据库成为瓶颈。
- 3、分布式锁：当多个请求几乎同时到达时，用 Redis 的 SETNX 命令抢锁，抢到锁的请求才能执行扣库存操作，其他请求等待或直接返回"已售罄"。同时给锁设置一个较短的过期时间，防止服务宕机后锁永远不释放。这种方式相当于在 Redis 层加了一道"闸门"，把并发请求串行化，虽然会牺牲一点性能，但能确保库存绝不会被超卖，适合对一致性要求极高的场景。

典型真题 (202605系分真题)

阅读以下关于在线教育系统容灾设计的叙述，在答题纸上回答问题 1~3。

某在线教育平台同时提供直播教学、点播教学、课程订购和学习记录管理等业务。平台采用微服务架构，核心数据存储使用 MySQL，部分热点数据和会话信息使用 Redis。

该平台主要业务包括：

- 1.直播授课：支持教师实时授课、学生在线观看、互动答疑等功能；
- 2.点播授课：支持学生观看录播课程；
- 3.课程订购：支持用户购买课程、订单支付和订购记录管理；
- 4.学习记录管理：记录学生课程学习进度、观看时长和学习历史。

其中，直播业务对实时性和连续性要求较高，要求：

RPO 不超过 5 分钟；

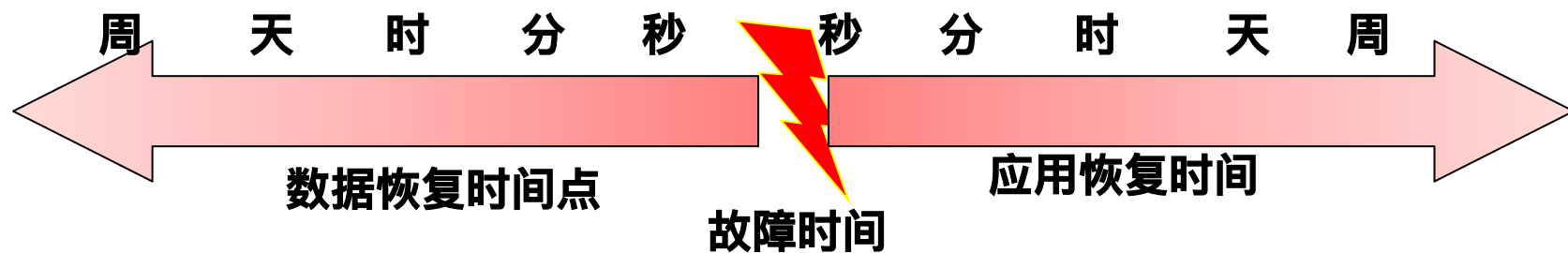
RTO 不超过 1 分钟；

点播业务对实时性要求较低，允许小时级故障恢复。

平台保存的数据包括用户信息、课程资源、订购记录、支付记录、直播互动数据、学习记录和统计日志等。项目组需要在冷备、热备、双活等容灾方案中进行选择，并比较同步复制、异步复制等数据复制方式，同时从成本、运维复杂度和定期演练必要性角度分析容灾建设方案。

典型真题 (202605系分真题)

灾备指标



◆数据恢复时间点(RPO)

◆RPO(Recovery Point Objective) :

- 以数据为出发点
- 能够容忍的数据丢失量

◆应用恢复时间(RTO)

◆RTO(Recovery Time Objective) :

- 以应用为出发点
- 能够容忍的应用系统恢复时间段

典型真题（202605系分真题）

【问题 1】（8 分）

请说明冷备、热备和双活三种容灾方案的区别，并结合 RPO 和 RTO 要求，分析直播业务和点播业务分别适合采用哪种方案。

参考答案：

冷备、热备和双活的区别如下：

容灾方案	特点	优点	缺点	适用场景
冷备	备用系统平时不承担业务，故障后再启动、恢复数据和切换	成本低，建设简单	RTO 较长，恢复速度慢	非核心业务、允许较长时间恢复的系统
热备	备用系统持续同步数据，平时处于待命状态，故障后快速切换	恢复较快，成本和复杂度适中	平时资源利用率不高，仍需切换时间	核心业务、对恢复时间要求较高的系统
双活	两个或多个站点同时对外提供服务，故障时流量快速切换	RTO 最短，可用性最高	成本高，运维复杂，数据一致性处理困难	对连续性要求极高的核心业务

直播业务要求 RPO 不超过 5 分钟、RTO 不超过 1 分钟，宜采用热备或双活方案。若追求极高连续性，可采用双活；若综合考虑成本，可采用热备并实现快速自动切换。点播业务允许小时级恢复，可采用冷备或异步备份等低成本方案。

典型真题 (202605系分真题)

【问题 2】 (8 分)

请比较同步复制和异步复制在以下方面的区别：

- 1.数据一致性；
- 2.系统性能；
- 3.网络依赖；
- 4.成本和适用场景。

并结合在线教育平台的业务特点，说明哪些数据适合同步或准同步复制，哪些数据适合异步复制。

典型真题 (202605系分真题)

对比维度	同步复制	异步复制
数据一致性	主站点写入后需等待备站点确认，数据一致性强，RPO 可接近 0	主站点提交后再异步传输到备站点，可能丢失最近一段时间数据
系统性能	会增加写入延迟，影响系统响应性能	对主营业务性能影响较小
网络依赖	对网络带宽、时延和稳定性要求高	网络要求相对较低，可容忍一定延迟
成本	通常成本较高，需要高质量链路和更复杂架构	成本较低，部署和维护相对简单
适用场景	订单、支付、账户等强一致性关键数据	日志、统计、学习记录、点播进度等允许短暂延迟的数据

同步复制一致性强，RPO 可接近 0，但会增加写入延迟，对网络质量要求高，成本较高。异步复制性能影响小、成本较低，但故障时可能丢失最近一段时间数据。在线教育平台应按数据重要性分级，订单、支付和订购关系等关键数据可采用同步或准同步复制，日志、统计、点播学习进度等可采用异步复制。

典型真题 (202605系分真题)

【问题 3】容灾建设因素分析 (9 分)

请从以下角度说明在线教育平台进行容灾建设时需要考虑的主要因素：

- 1.成本；
- 2.运维复杂度；
- 3.定期演练的必要性；
- 4.业务分级与容灾策略匹配。

参考答案：

容灾建设需要在业务连续性和建设成本之间平衡。双活可用性最高但成本和运维复杂度最高，热备成本和复杂度居中，冷备成本最低但恢复慢。容灾方案必须定期演练，以验证备份可用性、切换流程、人员响应和 RPO/RTO 指标是否达标。建议按业务分级设计容灾策略，直播核心链路采用热备或双活，点播和非核心统计业务采用冷备或异步备份。

作业

阅读下列说明，回答问题1至问题3,将解答填入答题纸的对应栏内。
某企业委托软件公司开发一套包裹信息管理系统，以便于对该企业通过快递收发的包裹 信息进行统一管理。在系统设计阶段，需要对不同快递公司的包裹单信息进行建模，其中， 邮政包裹单如图2-1所示。

- 【问题1】（14分）
请说明关系型数据库开发中，逻辑数据模型设计过程包含哪些任务?该包裹单的逻辑数据模型中应该包含哪些实体?并给出每个实体的主键属性。
- 【问题2】（6分）
请说明什么是超类实体?结合图中包裹单信息，试设计一种超类实体，给出完整的属性列表 。
- 【问题3】（5分）
请说明什么是派生属性，并结合图2-1的包裹单信息说明哪个属性是派生属性。

国内普通包裹详情单（通知单联） 邮1106

中国邮政 CHINA POST

PA05837748444

收件人	姓名: 电话:	内件品名及数量	接收局号码:
件	单位名称:		
人	详细地址:	是否保价 是 ☐ 否 ☐	寄件人名章:
寄件人	姓名: 电话:	保价金额: 元	重量: 克
	单位名称:	寄件人声明: 同意并遵守背面的“使用须知”, 如包裹无法投递, 按如下选择处理:	资费: 元
	详细地址:	☐ 退还寄件人 ☐ 抛弃处理	挂号费: 元
	用户代码: 邮政编码:	签字:	保价费: 元
		检查人员名章	回执费: 元
			总计: 元

PA 0583-7748 4 44 填写本单前, 务请阅读背面的“使用须知”! 您的签名意味着您理解并接受“使用须知”内容。

图 2-1 包裹单示意图

作业

【问题1】

逻辑数据模型设计过程包含的任务：

- (1)构建系统上下文数据模型，包含实体及实体之间的联系。
- (2)绘制基于主键的数据模型，为每个实体添加主键属性。
- (3)构建全属性数据模型，为每个实体添加非主键属性。
- (4)利用规范化技术建立系统规范化数据模型。

包裹单的逻辑数据模型中包含的实体：

- (1)收件人(主键：电话)。
- (2)寄件人(主键：电话)。
- (3)包裹单(主键：编号)。

【问题2】

超类实体是将多个实体中相同的属性组合起来构造出的新实体。 用户(姓名、电话、单位名称、详细地址)

【问题3】

派生属性是指某个实体的非主键属性由该实体其他非主键属性决定。

包裹单中的总计是由资费、挂号费、保价费、回执费计算得出，所以是派生属性。

作业

试题三 (25分)

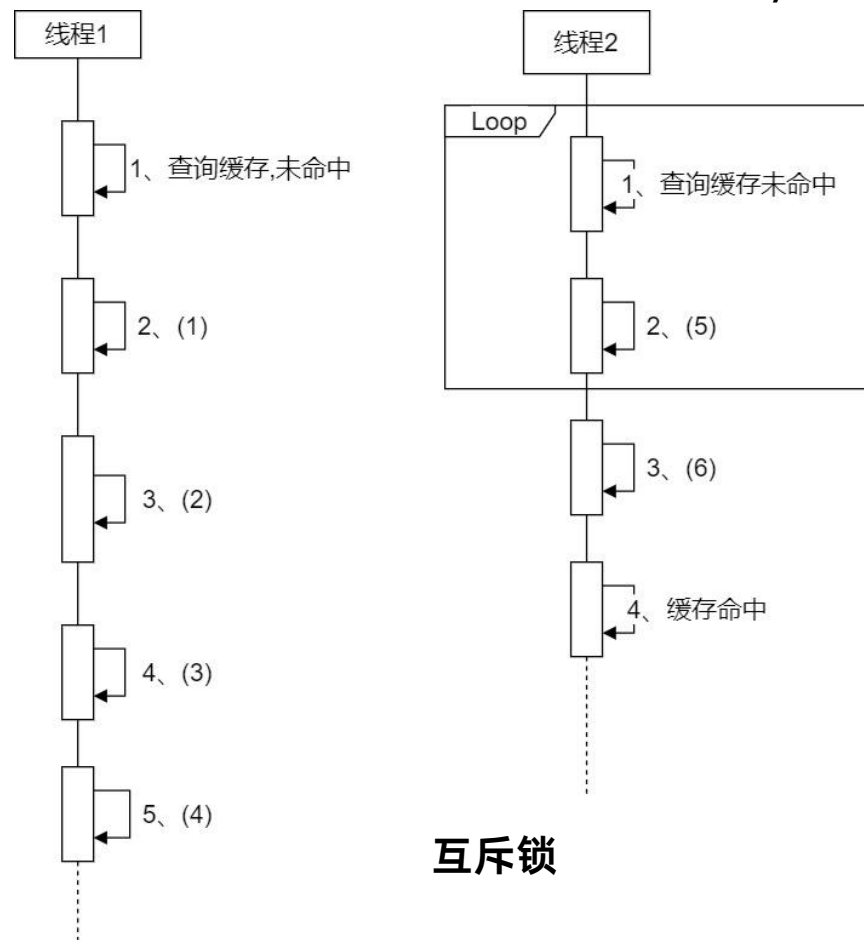
某电商平台为提升系统性能，采用MySQL作为主数据库，同时引入Redis作为缓存存储热点数据。系统主要支持商品信息查询、库存管理及订单存储等业务：商品详情页访问频繁，需快速响应；库存数据需实时准确，支持高并发修改；订单数据需长期保存，查询频率较低。

设计中，Redis主要存储热门商品基本信息（如名称、价格）和实时库存数量、热销商品排名；MySQL存储完整商品数据、库存明细、订单全量信息及用户数据。

【问题1】 (6分)

针对热点数据可能会发生的缓存失效问题公司架构师王工提出基于互斥锁的缓存实现方案。补充下面的架构图。

- (1) 获取互斥锁成功；
- (2) 查询数据库
- (3) 更新缓存
- (4) 释放锁
- (5) 获取互斥锁失败
- (6) 休眠一会，重试

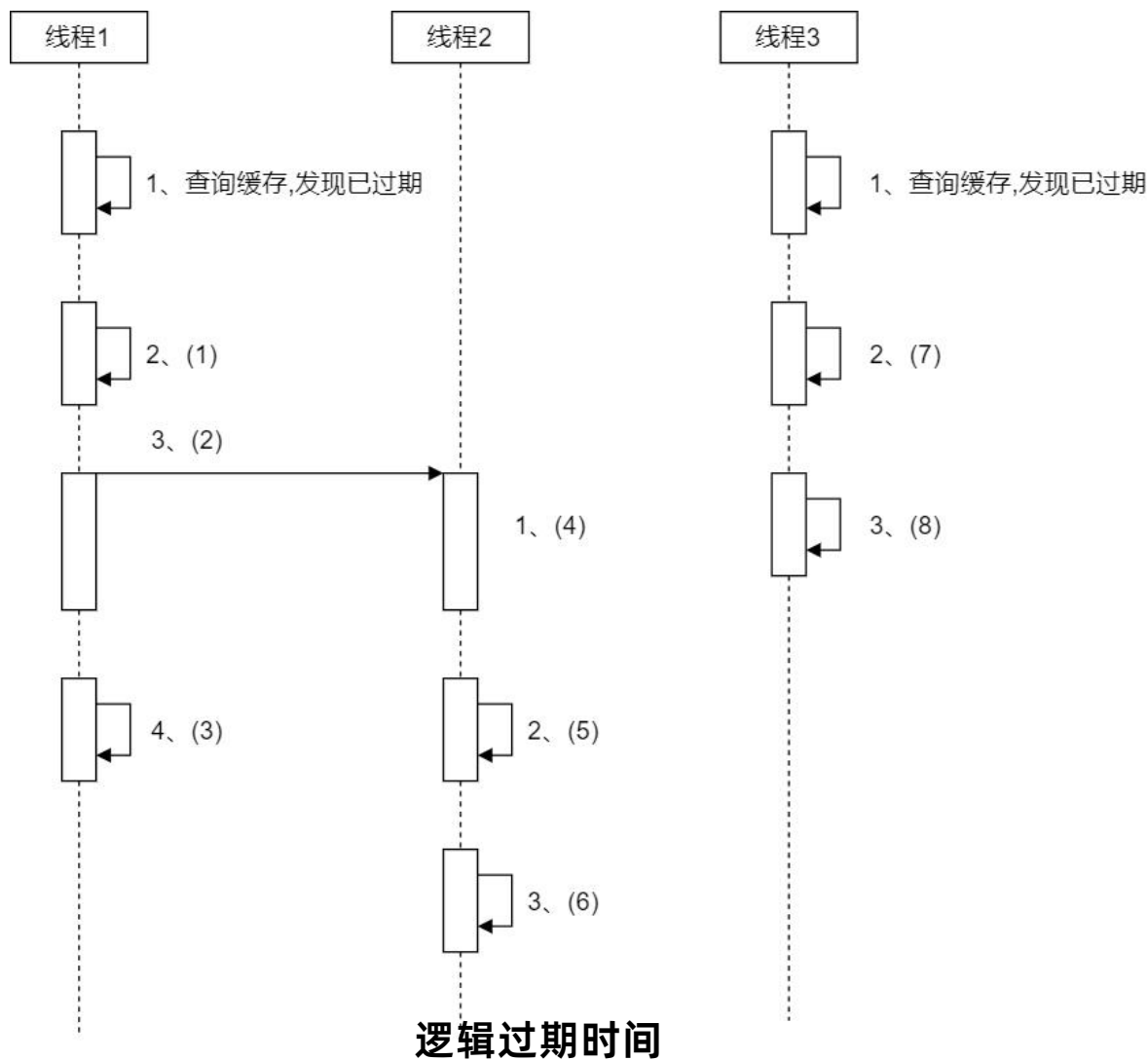


作业

【问题2】 (8分)

针对热点数据可能会发生的缓存失效问题公司架构师李工提出采用逻辑过期方案。补充下面的架构图。

- (1) 获取分布式锁成功
- (2) 开启新线程
- (3) 返回旧数据
- (4) 查询数据库
- (5) 更新缓存
- (6) 释放锁
- (7) 获取分布式锁失败
- (8) 返回旧数据



作业

【问题3】（11分）

对比王工和李工的方案各自的优缺点。

互斥锁方案的优点：

数据一致性高：互斥锁方案通过确保同一时间只有一个线程能够访问数据库并更新缓存，从而避免了多个线程同时访问数据库导致的脏数据问题，保证了数据的一致性。

实现简单直观：互斥锁方案逻辑清晰，易于理解和实现。开发者只需在缓存失效时加锁，获取数据后更新缓存并释放锁即可。

互斥锁方案的缺点：

线程阻塞：如果获取锁的线程更新缓存时间较长，其他线程会一直阻塞，影响系统响应时间。

作业

阅读以下关于数据管理的叙述，在答题纸上回答问题1至问题3。

【说明】

某软件企业开发了一套新闻社交类软件，提供常见的新闻发布、用户关注、用户推荐、新闻点评、新闻推荐、热点新闻等功能，项目采用MySQL数据库来存储业务数据。系统上线后，随着用户数量的增加，数据库服务器的压力不断加大。为此，该企业设立了专门的工作组来解决此问题。

张工提出对MySQL数据库进行扩展，采用读写分离，主从复制的策略，好处是程序改动比较小，可以较快完成，后续也可以扩展到MySQL集群，其方案如图4-1所示。李工认为该系统的诸多功能，并不需要采用关系数据库，甚至关系数据库限制了功能的实现，应该采用 NoSQL数据库来替代MySQL，重新构造系统的数据层。而刘工认为张工的方案过于保守，对该系统的某些功能，如关注列表、推荐列表、热搜榜单等实现困难，且性能提升不大；而李工的方案又太激进，工作量太大，短期无法完成，应尽量综合二者的优点，采用Key-Value数据库+MySQL数据库的混合方案。经过组内多次讨论，该企业最终决定采用刘工提出的方案。

【问题1】（8分）

张工方案中采用了读写分离，主从复制策略。其中，读写分离设置物理上不同的主/从服务器，让主服务器负责数据的（a）操作，从服务器负责数据的（b）操作，从而有效减少数据并发操作的（c），但却带来了（d）。因此，需要采用主从复制策略保持数据的（e）。

MySQL 数据库中，主从复制是通过 binary log来实现主从服务器的数据同步，MySQL 数据库支持的三种复制类型分别是（f）、（g）、（h）。请将答案填入（a）～（h）处的空白，完成上述描述。

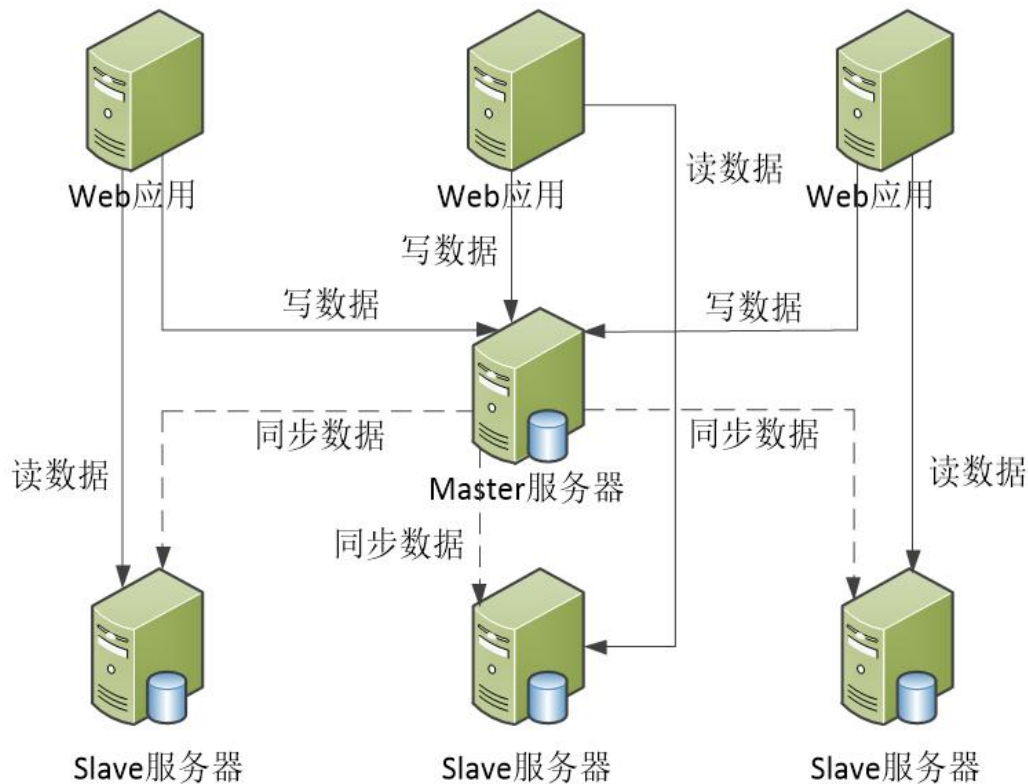


图 4-1 张工方案示意图

【参考答案】

(a) 写 (b) 读 (c) 延迟 (d) 数据冗余 (e) 一致性
(或同步) (f) 基于SQL语句的复制 (g) 基于行的复制
(h) 混合模式复制
注：(f) (g) (h) 不分次序。

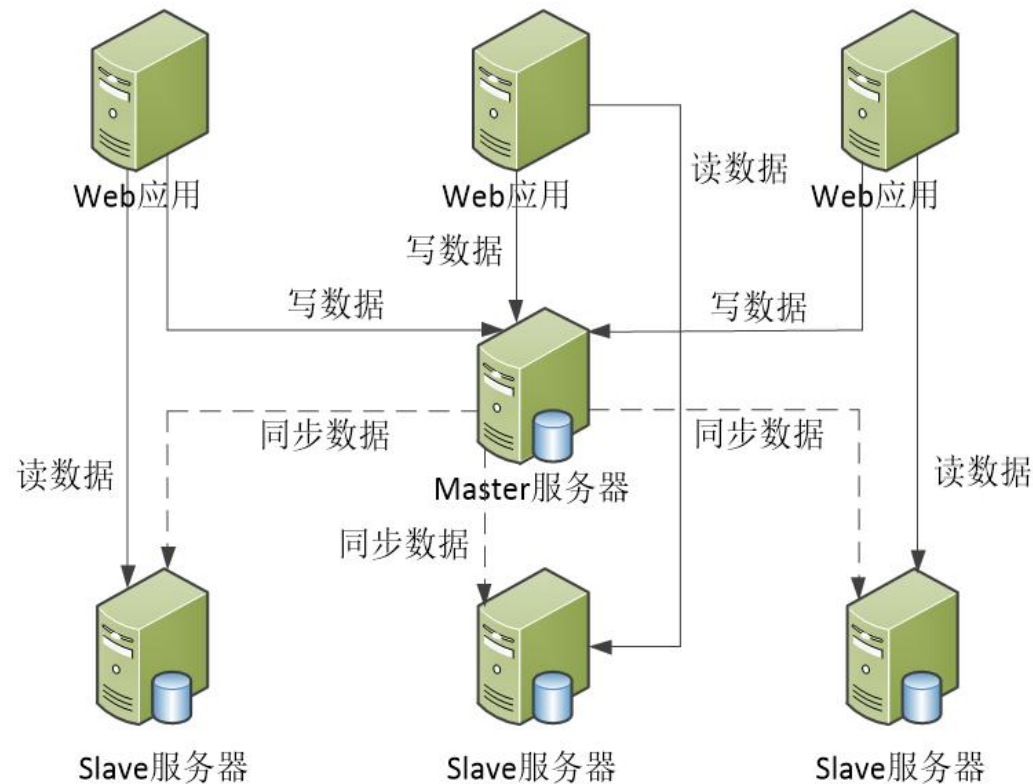


图 4-1 张工方案示意图

作业

【问题2】（3分）

李工方案中给出了关系数据库与NoSQL数据库的比较，如表4-1所示，以此来说明该新闻社交类软件更适合采用NoSQL数据库。请完成表 4-1 中的（a）~（d）处空白。

关系数据库与 NoSQL 数据库特征比较

表 4-1 关系数据库与 No SQL 数据库特征比较

特征	关系数据库	No SQL 数据库
数据一致性	实时一致性	(a)
数据类型	结构化数据	(b)
事务	高事务性	(c)
水平扩展	弱	强
数据容量	有限数据	(d)

作业

【参考答案】

- (a) 最终一致性
- (b) 非结构化数据
- (c) 软状态/柔性事务
- (d) 海量数据

典型真题

【问题3】（3分）

刘工提出的方案采用了Key-Value数据库+MySQL数据库的混合方案，是根据数据的读写特点将数据分别部署到不同的数据库中。但是由于部分数据可能同时存在于两个数据库中，因此存在数据同步问题。请用 200 字以内的文字简要说明解决该数据同步问题的三种方法。

典型真题

【问题3】（3分）

刘工提出的方案采用了Key-Value数据库+MySQL数据库的混合方案，是根据数据的读写特点将数据分别部署到不同的数据库中。但是由于部分数据可能同时存在于两个数据库中，因此存在数据同步问题。请用 200 字以内的文字简要说明解决该数据同步问题的三种方法。

【参考答案】

- (1) 采用自定义函数，进行MySQL数据库编程，利用触发器实现数据同步。
- (2) 采用消息中间件实现数据同步。
- (3) 采用同步工具，比如canal实现数据同步。

作业 (202605真题综合技术)

24、设关系模式 $R(A,B,C)$ 存在函数依赖 $A \twoheadrightarrow B$ 、 $B \twoheadrightarrow C$ ，该关系模式最高可达到 ()。

- A. 1NF
- B. 2NF
- C. 3NF
- D. BCNF

试题分析：

选项A，1NF（第一范式）是指关系模式中每个属性的值都是不可再分的原子值。本题A、B、C都是单一属性，每个字段只存一个值，天然满足1NF。

选项B，2NF（第二范式）是在满足1NF的基础上，要求所有非主属性都完全依赖于候选键，不能只依赖候选键的一部分（即消除部分依赖）。部分依赖只可能发生在候选键是复合键的情况下。本题中候选键是单属性A，不存在“只依赖一部分”的可能。B和C都是被A完整决定的，不存在部分依赖，满足2NF。

选项C，3NF（第三范式）是满足2NF的基础上，要求非主属性之间不能存在传递依赖。也就是说，非主属性必须直接依赖于候选键，不能通过另一个非主属性间接依赖。本题中 $A \rightarrow B$ ， $B \rightarrow C$ ，所以C是通过B间接依赖于A的，形成了 $A \rightarrow B \rightarrow C$ 的传递依赖。非主属性C传递依赖于候选键A，违反3NF。

选项D，BCNF（巴斯-科德范式）是在满足3NF的基础上，要求每一条函数依赖的左部都必须是候选键。也就是说，能决定其他属性的那个属性，本身必须能唯一标识一行。BCNF不仅排除了任何属性（主属性和非主属性）对候选键的部分依赖和传递依赖，而且排除了主属性之间的传递依赖。本题中依赖 $B \rightarrow C$ 中，左部是B，但B不是候选键。B不能唯一标识一行，却能决定C，违反BCNF。

参考答案：B

作业 (202605真题综合技术)

30、设关系 $R(A,B)$ 有 5 个元组， B 为 R 的主键，关系 $S(B,C)$ 有 10 个元组。 R 与 S 进行自然连接时， R 与 S 自然连接结果元组数的可能范围是 ()。

- A. $[0,10]$
- B. $[0,5]$
- C. $[5,10]$
- D. $[10,50]$

试题分析：

设关系 $R(A,B)$ 含 5 个元组， B 为主键，故 R 中 B 的 5 个取值互不相同；关系 $S(B,C)$ 含 10 个元组， B 非主键，故 S 中 B 的取值允许重复。自然连接以公共属性 B 为连接条件，将 R 与 S 中 B 值相等的元组进行拼接，结果元组数取决于 R 中各 B 值在 S 中的匹配次数之和。这里存在两种极限情况：

若 R 中 5 个互异的 B 值均未在 S 中出现，则无一匹配，结果为 0 个元组。

若 R 中每个 B 值均在 S 中出现，由于 S 共有 10 个元组，即使每个 R 元组匹配多个 S 元组，匹配总数也不可能超过 S 的元组总数，故结果最多为 10 个元组。

综上所述， R 与 S 自然连接的结果元组数的可能范围为 $[0, 10]$ 。

参考答案：A

作业（202605真题综合技术）

34、实体关系模型中，如果一个集合中的实体与同一集合内的多个实体存在联系，则该实体被称为（ ）。

- A. 复合实体
- B. 关联实体
- C. 弱实体
- D. 视图

试题分析：

实体-关系模型中的实体，按不同维度可以分为以下几类：

按存在独立性划分：

强实体（普通实体）：有独立主键，可独立存。例如学生、课程。

弱实体 无独立主键：必须依赖强实体才能存在。弱实体没有独立主键，必须借助强实体的主键才能唯一标识自己，所以标准物理设计中必须独立成表，且外键参与自身主键。例如订单明细需要依赖订单这个强实体才能存在，那么订单明细表的主键就是订单号+序号，其中订单号是外键。但是当弱实体与强实体是 1:1 关系，且业务上永远绑定在一起时，可以合并到强实体表中，不单独建表。假设每单只有一条明细的话，可以合并到订单表，加一个字段即可。

按联系方式划分：

关联实体：解决两个不同实体集之间M:N关系的中间实体。例如，选课实体就关联了学生实体与课程实体。

递归实体（复合实体）：指的是同一实体集内部实体之间存在联系。例如，经理和下属都是"员工"这个实体集中的实体，两者存在关联关系——经理管理下属。

按层次关系划分：

超类实体：泛化后的上层实体，例如，用户。

子类实体：特化后的下层实体，例如，教师、学生都是用户的子类。

参考答案；A

作业 (202605真题综合技术)

54、Elasticsearch服务器依赖倒排索引作为核心数据结构，其主要作用是（ ）。

- A. 加快文档内容搜索
- B. 提高数据写入速度
- C. 保障集群节点间数据同步
- D. 实现数据库事务的一致性

试题分析：

倒排索引是全文检索的核心基石。文档写入时先分词，然后建立"关键词 → 包含该关键词的文档ID列表"的映射关系。查询时，直接通过关键词定位文档，无需逐行扫描全部数据，将检索复杂度从 $O(n)$ 降到接近 $O(1)$ ，从而实现毫秒级的文档内容搜索。

参考答案：A

作业 (202605真题综合技术)

65、关系型数据库中表与表间的实体联系通过实体的（ ）实现。

- A. 公共索引
- B. 公共元组
- C. 公共属性
- D. 公共列族

试题分析：

在关系型数据库中，表与表之间的实体联系是通过公共属性实现的。所谓公共属性，就是两个表中都包含的相同字段，通常表现为一个表的外键引用另一个表的主键。例如学生表和成绩表都含有"学号"这一属性，通过该公共属性即可将两张表的数据关联起来，从而建立起学生与成绩之间的实体联系。公共索引仅用于加速查询，公共元组指的是表中的行数据，公共列族则是列式数据库的概念，均与表间联系的实现无关。故选C。

参考答案：C

作业 (202511真题综合技术)

数据库 $R-(R-S)$ 与下列哪项等价 ()。

- A. $R \cap S$
- B. $R \cup S$
- C. $R-S$
- D. $R \times S$

参考答案：A

试题分析

根据题干首先进行 $R-S$ 的运算，求得的结果是属于R但是不属于S的元组的集合，然后再与R进行差运算，就可以得出R与S共有的元组的集合，即等于 $R \cap S$ 。这里举一个例子，

如：

R			S		
A	B	C	A	B	C
1	2	3	1	2	3
4	5	6	7	8	9

进行 $R-(R-S)$ 运算后的得到的结果是123。

选项B进行 $R \cup S$ 运算，得到属于R或属于S的元组构成的集合，对应上述例子结果是123,456,789。

选项C进行 $R-S$ 运算，得到属于R但不属于S的元组构成的集合，对应上述例子结果是456。

选项D进行 $R \times S$ 运算，若R有u个元组，S有v个元组，则 $R \times S$ 有 $u \times v$ 个元组，具体对应上述例子就是123123,123789,456123,456789

作业（202511真题综合技术）

SELECT查询结果的本质是（）。

- A.元组
- B.集合
- C.属性值
- D.序偶

参考答案：B

试题分析

SELECT查询语句的标准格式是：

SELECT[ALL|DISTINCT]<目标列表达式>[,<目标列表达式>]...

FROM<表名或视图名>[,<表名或视图名>]

[WHERE<条件表达式>]

[GROUP BY<列名1>[HAVING<条件表达式>]]

[ORDER BY<列名 2>[ASC|DESC]...]

其中，SELECT和FROM是必须的。SELECT子句对应的是关系代数运算中的投影运算，用来列出查询结果中的属性。ALL 显示所有行（默认选项）。DISTINCT 去除重复的元组（行），只返回唯一的记录。FROM指定要查询的数据来源（表或视图）。WHERE设置过滤条件，用于筛选元组。GROUP BY将数据按指定列进行分组，常与聚合函数（如COUNT, SUM）一起使用。HAVING只能与GROUP BY搭配，对分组后的结果进行过滤。ORDER BY用于对最终结果集按指定列进行排序。

SELECT查询结果的本质是一个集合。这是因为关系数据库的理论基础是关系模型，其中查询结果被视为一个关系，而关系是元组（行）的集合。即使查询结果只包含一行，它仍然是一个集合（单元素集合）。题干选项A，元组可以看作是查询结果中的一行。

选项B，集合是整个查询结果（多行元组构成的一个集合）。

选项C，属性值是查询结果中的某一列的值。选项D，序偶特指一个二元组，即只有两个值的有序对。选项ACD表述过于片面，只有选项B合理。

作业（202511真题综合技术）

数据库设计中，产出关系模型的阶段是（）。

- A.概念设计
- B.物理设计
- C.逻辑设计
- D.需求分析

参考答案：C

试题分析

数据库设计主要分为6个阶段：

- ① 用户需求分析：分析用户活动、收集数据确定系统边界。
- ② 概念结构设计：将应用对象抽象成不依赖于数据库管理系统DBMS的数据模型，即概念模型。描述概念模型的较理想的工具是实体-关系（E-R）图。
- ③ 逻辑结构设计：将抽象的概念模型转化为与选用的DBMS 产品所支持的数据模型相符合的逻辑模型，即关系模式。
- ④ 物理结构设计：为逻辑数据模型选取一个最适合应用环境的物理结构（包括存储结构和存取方法），即在物理设备上的实现方案。
- ⑤ 数据库实施阶段：根据前两个阶段的结果建立数据库，编制与调试应用程序，组织数据入库，并进行试运行。
- ⑥ 数据库运行和维护阶段：在数据库系统正式投入运行后，对其进行长期的评价、调整与修改。

作业（202511真题综合技术）

两阶段提交协议中，事务获取共享锁（S锁）后，对数据的操作权限是（）。

- A.只能读不能写
- B.不能读不能写
- C.能读能写
- D.不能读能写

参考答案：A

试题分析

两阶段提交协议分为准备阶段和提交/回滚阶段，这是一种在分布式数据库系统或分布式事务中使用的算法，其核心目标是确保所有参与分布式事务的节点要么全部提交事务，要么全部回滚事务，从而维护数据的强一致性。

两阶段提交协议通常包含两种角色：

协调者：负责驱动整个提交流程，并做出最终决定。

参与者：分布式系统中所有涉及该事务的数据库或服务节点。它们各自管理自己的数据，并执行事务操作。

在准备阶段，协调者向所有参与者发送一条“准备”消息。每个参与者在收到“准备”消息后，会执行本地事务、记录日志等操作。如果所有准备工作都成功，参与者会回复协调者一个“同意”消息。

在提交/回滚阶段，协调者根据第一阶段的回复结果，做出全局决策并通知所有参与者。这里分为两种情况：

情况一，所有参与者都回复“同意”，协调者向所有参与者发送“提交”命令。参与者在收到“提交”命令后，正式将本地事务提交，永久性地更新数据，并释放所有锁。参与者完成提交后，向协调者发送“已完成”的确认消息。

情况二，任何一个或多个参与者回复“中止”，或协调者在规定时间内未收到某个参与者的回复，协调者向所有参与者发送“回滚”命令。参与者在收到“回滚”命令后，利用之前写入的撤销日志来回滚整个事务，撤销所有本地操作，并释放所有锁。参与者完成回滚后，向协调者发送“已回滚”的确认消息。

在每个参与者在执行本地事务的操作时，都需要依赖数据库本身的排他锁和共享锁机制。

排他型封锁（简称 X 封锁、写锁）：如果事务 T 对数据 A（可以是数据项、记录、数据集，乃至整个数据库）实现了 X 封锁，那么只允许事务 T 读取和修改数据（写数据）A，其他事务要等事务 T 解除 X 封锁以后，才能对数据 A 实现任何类型的封锁。

共享型封锁（简称 S 封锁、读锁）：如果事务 T 对数据 A 实现了 S 封锁，那么允许事务 T 读取数据 A，但不能修改数据（写数据）A，在所有 S 封锁解除之前绝不允许任何事务对数据 A 实现 X 封锁。

本题虽然有两阶段提交协议，但是本质上考的是共享锁和排他锁的特点。当事务获取共享锁（S锁）后，对数据的操作权限是只能读不能写。

作业（202505真题综合技术）

7. 关系代数运算不包括（）。

A.选择 B.投影 C.删除 D.连接

【解析】：

关系代数运算符有4类：集合运算符、专门的关系运算符、算术比较符和逻辑运算符。

【参考答案】：C

32.数据库三级模式中，表示用户局部数据的是（）。

A.内模式 B.外模式 C.概念模式 D.逻辑模式

【解析】：

外模式也称用户模式或子模式，是用户与数据库系统的接口，是用户需要使用的部分数据的描述。

它由若干个外部记录类型组成。用户使用数据操纵语言对数据库进行操作，实际上是对外模式的外部记录进行操作。

【参考答案】：B

作业（202505真题综合技术）

33.数据库三级模式中，描述记录间联系、数据完整性、安全性的是（）。

A.存储模式 B.概念模式 C.内模式 D.外模式

【解析】：

概念模式也称模式，是数据库中全部数据的逻辑结构和特征的描述，它由若干个概念记录类型组成，只涉及“型”的描述，不涉及具体的值。概念模式的一个具体值称为模式的一个实例，同一个模式可以有很多实例。概念模式反映的是数据库的结构及其联系，所以是相对稳定的；而实例反映的是数据库某一时刻的状态，是相对变动的。

需要说明的是，概念模式不仅要描述概念记录类型，还要描述记录间的联系、操作、数据的完整性和安全性等要求。但是，概念模式不涉及存储结构、访问技术等细节。只有这样，概念模式才算做到了“物理数据独立性”。

【参考答案】： B

作业 (202505真题综合技术)

34. 已知关系 $R(a,b,c,d)$ 和 R 上的函数依赖 $F=(a \rightarrow cd, c \rightarrow b)$, 则 R 的候选码是 ()。

A.a B.b C.c D.d

【解析】：

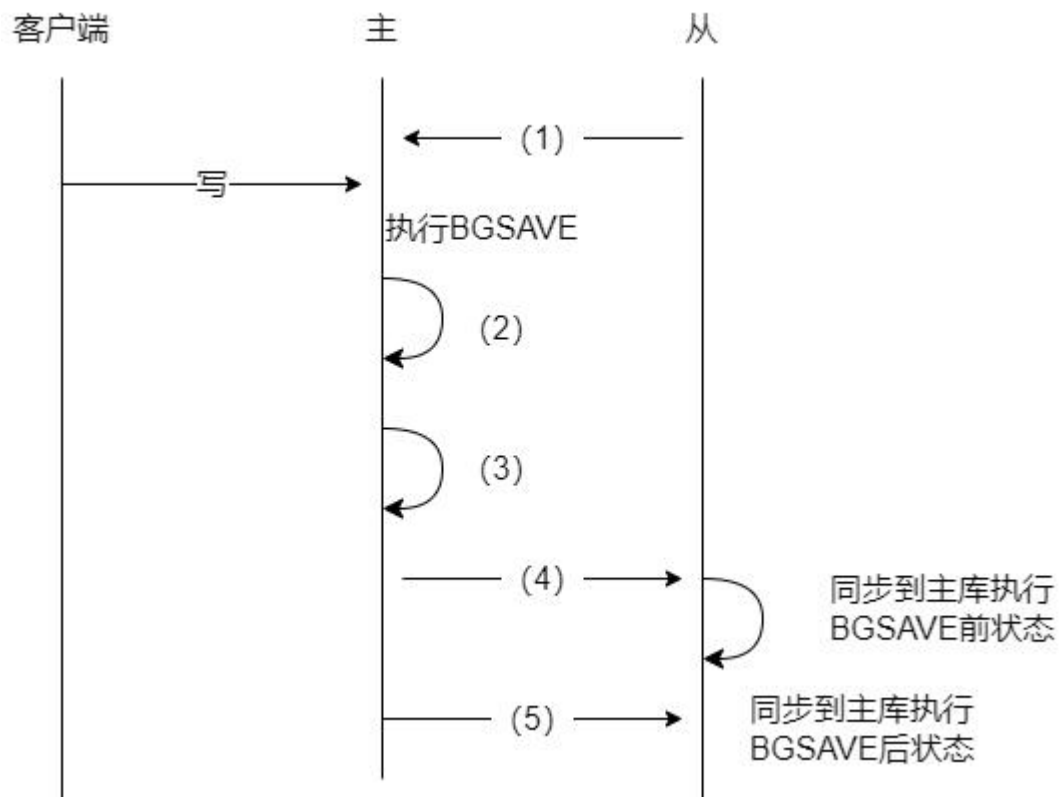
已知 $a \rightarrow cd$, 推导出 a 可确定 c 和 d ;

进一步通过 $c \rightarrow b$, 可得 $a \rightarrow b$, 因此闭包为 $a = U$ (全体属性), 候选码是 a 。

【参考答案】： A

作业 (202505真题)

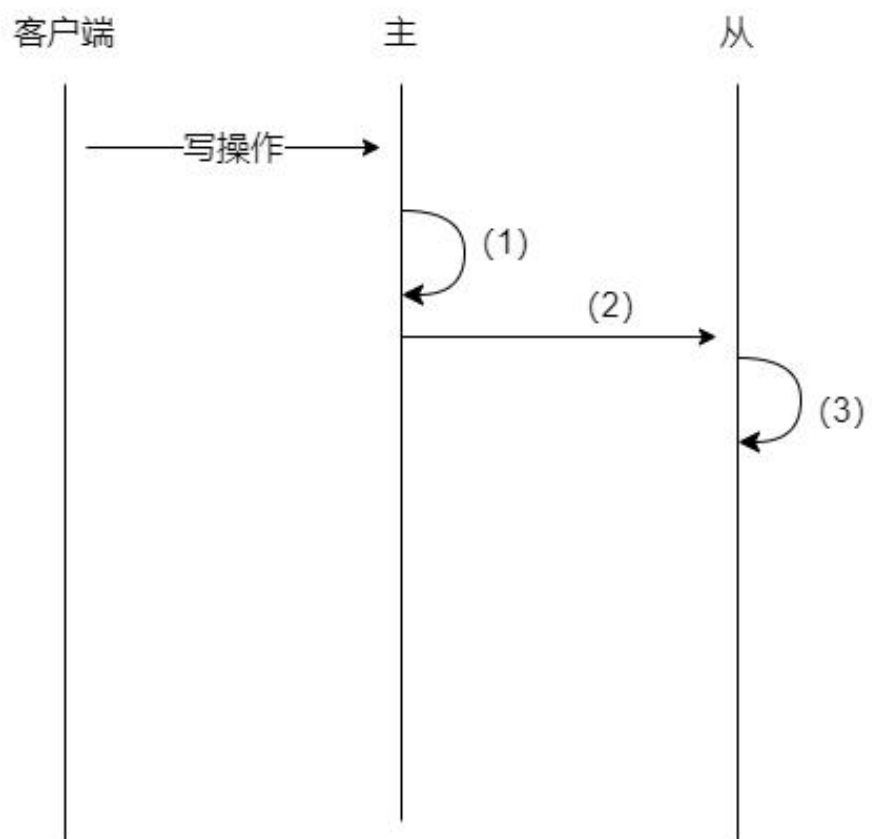
1、(10分) redis 主从同步，按要求填空，首次全量从库同步主库过程。



作业 (202505真题)

2、(6分) 增量主从库同步。

增量主从库同步



作业 (202505真题)

3、redis数据库持久化的方式，列举两种并作比较，说明两种持久化方式的优缺点。（9分）

作业答案

1 【参考答案】：

- (1) 请求数据同步
- (2) 判断是否全量同步
- (3) 将操作记录在repl_baklog内
- (4) 发送RDB文件
- (5) 发送repl_baklog中的命令

【解析】：

数据同步阶段（含 BGSAVE 操作）

(1) 同步请求：从节点发送psync或psync2命令给主节点，请求进行数据同步。初次同步时，从节点发送的psync命令中，replid和offset的默认值分别是?和-1，表示需要进行全量复制。

(2) 主节点执行 BGSAVE：主节点接收到从节点的同步请求后，判断需要进行全量复制，会 fork 一个子进程来执行BGSAVE命令。BGSAVE用于在后台生成 RDB 快照文件，这个文件保存了主节点当前时刻的数据状态。在执行BGSAVE期间，主节点会开辟一个命令缓冲区，用来记录在此期间接收到的写操作指令。

(3) 生成并发送 RDB 文件：BGSAVE子进程生成 RDB 文件后，主节点通过 socket 连接将 RDB 文件发送给从节点。

(4) 从节点接收并处理 RDB 文件：从节点接收到 RDB 文件后，会清空自己以前的数据，然后执行 RDB 文件的恢复过程，将自身数据库状态更新至主节点执行BGSAVE前的状态。

(5) 同步写命令缓冲区：从节点完成 RDB 文件恢复后，会向主节点发送信息告知恢复已完成。主节点将在执行BGSAVE期间记录在命令缓冲区中的写命令，以 AOF（Append Only File）的格式发送给从节点。从节点接收这些命令并执行，使其数据库状态同步到主节点执行BGSAVE后的状态。如果从节点开启了 AOF 持久化功能，它会进行bgrewriteaof操作，生成最新的 AOF 文件，以保证数据的持久性和安全性。

作业答案

2 【参考答案】：

- (1) 检查数据更改
- (2) 发送offset之后的数据
- (3) 执行命令

作业答案

3 【参考答案】：

1. RDB (Redis Database)

优点：

高性能：仅在触发快照时 fork 子进程进行持久化，不影响主进程性能。

恢复快：完整二进制文件，重启加载速度显著快于 AOF。

节省空间：紧凑二进制格式，适合备份、灾难恢复。

缺点：

数据易丢失：默认触发频率为 5 分钟一次（可配置），若在此期间宕机，可能丢失最新数据。

fork 开销：大数据量下 fork 子进程可能导致短暂阻塞（毫秒级）。

2. AOF (Append Only File)

优点：

数据安全性高：支持秒级持久化（everysec 策略），最多丢失 1 秒数据。

实时记录：记录写操作命令，可通过回放日志恢复数据。

兼容性强：即使日志文件损坏，也可通过 redis-check-aof 工具修复。

缺点：

文件体积大：记录所有写命令，文件增长速度快于 RDB。

性能开销：频繁写磁盘（尤其 always 策略）可能降低 QPS。

恢复较慢：需逐条执行命令回放，大数据量下耗时较长。

对比选择建议：

优先选 RDB：若业务对数据丢失容忍度高（如缓存场景），追求高性能恢复。

优先选 AOF：若需保证数据完整性（如支付、账户系统），可接受稍低性能。

最佳实践：二者同时启用，兼顾数据安全性与恢复效率。

作业答案

Redis 提供两种主要的持久化机制（ps：持久化是为了快速的恢复数据而不是为了存储数据）：

持久化机制	说明	优点	缺点	适用场景
RDB (Redis Database)	在指定的时间间隔内，将内存中的数据集快照写入磁盘，恢复时将快照文件直接读到内存。	1. 适合大规模数据恢复 2. 对性能影响小，fork子进程进行持久化，主进程继续处理请求 3. 文件紧凑，适合备份和灾难恢复	1. 可能会丢失最后一次快照后的数据 2. fork子进程时，内存不足可能影响性能 3. 保存RDB文件时，如果数据集大，时间会较长	1. 可以容忍一定数据丢失 2. 大规模数据恢复场景 3. 对数据恢复速度要求较高
AOF (Append Only File)	以日志形式记录每个写操作，追加到AOF文件，重启时重新执行这些命令恢复数据。	1. 数据安全性高，可配置不同同步策略，最多丢失1秒数据 2. AOF文件是有序的写操作日志，易读，可修改修复	1. AOF文件通常比RDB文件大 2. 性能比RDB稍差，因为要不断记录写操作 3. 恢复速度比RDB慢	1. 对数据安全性要求高，不能容忍数据丢失 2. 对恢复速度要求不苛刻
RDB + AOF (版本4.0新增的机制)	同时使用RDB和AOF持久化。	兼具RDB和AOF的优点，既有RDB的快速恢复能力，又有AOF的数据安全性。	1. 占用更多磁盘空间 2. 配置和管理相对复杂	对数据安全性和恢复速度都有较高要求的场景

THANKS

 极客时间 | 训练营